



ClaimSphere[®] UDA

Universal Data Adaptor

March 2021

This document contains proprietary information.

© Cognizant Technology Solutions 2020. All rights reserved.



TABLE OF CONTENTS

1. About UDA	3
1.1. Intended Audience	3
1.2. Scope of UDA	3
2. UDA Workflow	4
2.1. UDA Workflow - Description	4
3. Getting Started – UDA	8
3.1. Prerequisites	8
3.2. UDA Source Staging Table Setup	9
3.2.1. Source Mode as File(F) – Incumbent Vendor Layout	9
3.2.2. Source Mode as Table(T) – Client DB/DW	10
3.3. Source DB – File Configuration	10
3.3.1. SRC_FILE_MASTER	11
3.3.2. SRC_LOAD_MASTER	11
3.3.3. SRC_TABLE_MASTER	11
3.4. CS File Master Configuration:	12
3.5. CS Table Configuration:	15
3.6. CS Table Master Configuration:	16
3.7. Client Layout – CS Layout Mapping Configuration:	17
3.7.1. CS_STG_Proj_Layout	18
CS_MAP_Reference	19
CS_MAP_CONDITION	20
3.7.2. CS_STG_PROJ_MAPPING	21
3.8. Output File Configuration	22
3.8.1. Custom Procedure Configuration	23
3.9. Package Configuration	23
3.9.1. Process Status & Rerun of Failed Packages	24
Steps to be followed for rerun:	24
3.9.2. Data Cleanup & Archival:	25
3.10. General Setup	25
3.11. Error Reports	26
3.12. Control File	26
3.13. Source Audit Report	27
3.14. Claims Count Report	27
4. System Requirements	28
5. UDA Application Setup	28
5.1. Prerequisites	28
5.2. Create Environment Folder	28
5.2.1. Configuring UDA_DLA_Master.dtsConfig, DLA_Slave.dtsConfig	29
5.2.2. Configuring UDA_DLA_Slave_TableLoad.dtsConfig:	29
5.3. Create Database	29
5.4. SQL Objects Deployment	30
5.5. SSIS Packages Deployment	30
5.6. Execution - UDA Application	31
5.7. Job Progress	31
6. Annexure	32
7. Glossary	32

1. About UDA

Universal Data Adaptor (UDA) is an implementation tool, designed to help the clients to migrate/adopt ClaimSphere application effortlessly and swiftly to generate NCQA HEDIS rates and other State measure rates as specified by NCQA and other regulatory bodies for health plans for annual Quality Submission and Compliance.

This is an add on application to assist customers to deliver the required data files to ClaimSphere application with no changes from the client side during new client implementation. The clients can deliver the data intake files in their system generated layout OR provide the DB/DW tables that holds the data required for arriving at incumbent vendor layout, UDA translates them to the required ClaimSphere layout for Measure Rate Generation.

UDA largely aims to reduce the time and effort involved in source data file preparation and reducing the dependency on customers, driving the layout conversion through configuration and mapping rules set in the application. During onboarding of new clients, existing source layouts or existing DB/DW can be used as such instead of waiting for the client's data extraction software updated to produce intake files in ClaimSphere layout, thereby expediting the implementation timelines.

This document provides all information and instructions that is required to setup data intake files of a new client transferring to ClaimSphere, system requirements and application installation guidelines.

1.1. Intended Audience

UDA is intended for the following users:

- ClaimSphere Product Consultants, Technical team involved in implementing ClaimSphere application for a new client.
- ETL / Business team involved in mapping source data to ClaimSphere intake files.

1.2. Scope of UDA

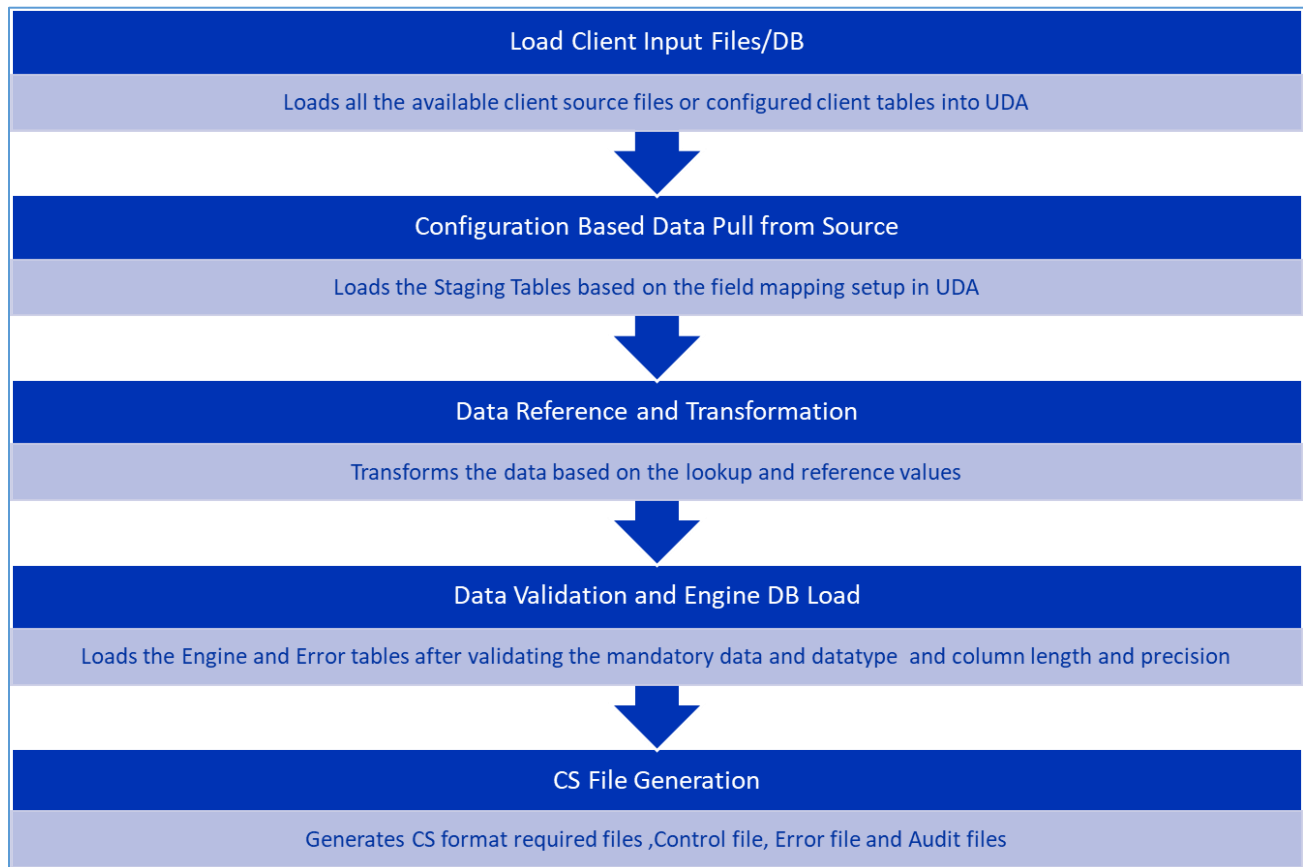
The scope of UDA is:

- Source file conversion from incumbent vendor format to ClaimSphere HEDIS intake files.
- ClaimSphere HEDIS intake file generation through connection with client environment DB/DW.
- Minimal Data type validation and minimal transformation to achieve ClaimSphere format.
- Supporting multiple versions (current year NCQA version, previous or future year's version) as per client's implementation timeline and request.
- Not a replacement tool for HEDIS.

2. UDA Workflow

For a Healthcare Client, migration to new HEDIS tool often involves changes to their data extraction software. Extraction layouts used for an existing HEDIS tool becomes useless when they transfer to a new HEDIS tool. When the data extracts at client end is being converted to ClaimSphere format, it involves laborious discussions with the source ETL team, Source Business team, Product team and account consultant teams that could go on for almost 6 to 8 months, delaying the onboarding process in entirety.

The UDA application aims to navigate some of these challenges for the new clients adopting ClaimSphere Product and help the technical team / ClaimSphere product consultants to avoid the longer implementation timeline to start using the ClaimSphere application while the client data extracts are still converted to the CS format. UDA significantly improves the Product adoption time.



2.1. UDA Workflow - Description

1	One – Time Configuration setup in UDA mapping DB
	The preference of using existing data files from clients in incumbent vendor layout or using the client DB/DW tables that contains data required for CS layout is collected from the client and UDA application is setup accordingly. All the files/tables and their layouts are identified & configured during the implementation phase. • The key fields and joining fields in each of these files are obtained through a mapping document from the client side POC. • A Mapping document containing ClaimSphere fields for each of the intake layouts and their definition is shared with the Client and corresponding source side mapping field information is collected. • Configuration tables like Table level mapping, Layout level mapping (CS_STG_PROJ_LAYOUT, CS_STG_PROJ_MAPPING, CS_MAP_REFERENCE, CS_MAP_TRANSFORM, and CS_MAP_CONDITION) are set up with information from the mapping document. • Lookup values and Transformation values are setup in tables like CS_CODE_LOOKUP, CS_MAP_TRANSFORM. • Configuration for supporting applications other than QAAS/HEDIS is also setup. • Configuration on the ClaimSphere intake file version to be supported is set up and corresponding ClaimSphere Layout tables are created in the application. • Configuration for the Source mode as files or Tables is setup in ETL_CONFIG table. • Once all the configurations are finalized and sample values are reviewed, the UDA application is installed and dry run version is signed off.
2	Load incumbent vendor files/ client DB/DW tables
	Depending on source mode as Client data files (incumbent vendor layout) or Client tables containing information related to health plan members, providers and claims to compute HEDIS and State Measure rate are configured as part of one time setup. All the source layouts are identified during the implementation phase and are setup to be transferred to the pre-defined server location (if source is file) or connection manager & string (if client tables) to be consumed by UDA. • The first step in the Data Load conversion is to load the source files or tables as such through the UDA into the SRC_* (Source Staging DB) setup inside the application. • Records with complete data in the identified source layout ensure smooth load process into Source DB. • Files or records within files with incorrect layout would be classified as erroneous records/files and is logged within the application and generated as error reports. • All the source files are loaded in parallel. • No limit on the number of files/tables for each run. • Any control files transmitted along with the source files will also be consumed.
3	Configuration Based Data Pull from Source DB
	Configuration based data pull is driven through the mapping between the source fields in each layout to corresponding field in ClaimSphere formatted files. The mapping DB configuration for each ClaimSphere intake file type is a one-time process setup by technical team during implementation.

- The data from the source DB is pulled for each ClaimSphere layout according to the configuration DB – CS_STG_PROJ_LAYOUT and CS_STG_PROJ_MAPPING.
- If a ClaimSphere file type demands data from multiple client files/tables, various UDA staging tables are joined according to the pre-defined join conditions and join keys from CS_MAP_REFERENCE setup table.
- If there is any need for a specific filter condition to be applied while extracting the data from the staging table(s), CS_MAP_CONDITION holds any customizable filter conditions.
- Once the dynamic / mapping based source staging tables and their join and filter conditions are identified, data is pulled into the CS_STG_* tables for each ClaimSphere file types.

For more information on the different ClaimSphere intake file types and their layout, refer to HEDIS Input File Specification document attached in Annexure.

4 Data Reference and Transformation

Data Reference and Transformation is a 2 – stage process that involves the following.

1	Lookup & Reference Updates	Lookup & Reference updates certain data fields that needs to be translated to CS format based on the pre-defined lookup values. Example - Data Source Type in the source field may contain the values “PS”, “SS”, “NS” representing Primary Source, Standard Supplemental source and Non Standard Supplemental source. ClaimSphere accepts the values “01”, “02”, “03” to represent Primary Source, Standard Supplemental source and Non Standard Supplemental source. UDA converts values in Data Source Type to CS accepted values based on the below mapping setup in CS_CODE_LOOKUP <table border="1"> <thead> <tr> <th>Source Side Value</th><th>CS Side value</th><th>Description</th></tr> </thead> <tbody> <tr> <td>PS</td><td>01</td><td>Admin Claim</td></tr> <tr> <td>SS</td><td>02</td><td>Standard Supplemental</td></tr> <tr> <td>NS</td><td>03</td><td>Non Standard Supplemental</td></tr> </tbody> </table> Example – Gender could be stored as 1,2,3 or Male, Female or Unknown in source and UDA converts it to “M”, “F”, “U”.	Source Side Value	CS Side value	Description	PS	01	Admin Claim	SS	02	Standard Supplemental	NS	03	Non Standard Supplemental
Source Side Value	CS Side value	Description												
PS	01	Admin Claim												
SS	02	Standard Supplemental												
NS	03	Non Standard Supplemental												
2	Transformation	Data Transformation process converts incoming source data/type into CS expected data/type/length based on the pre-defined transformation values identified and setup during the implementation.												

Example: Member Name in the source files is not stored as First Name, Middle Name and Last Name, but as a single 200 byte field.UDA applies the transformation logic to split the Member Name to First name, Last Name and Middle name according to the predefined transform rule that specifies the position, length of the split required. Example: Email Id field is not a required field in Client Member record, whereas ClaimSphere requires Email Id as the mandatory field. UDA can apply the default email id to records that does not have an email id and retain the same Email Id in the records that client sends through pre-defined transform rules. <table border="1"> <thead> <tr> <th>CS field that requires transformation</th><th>Transform Rule</th><th>Description</th></tr> </thead> <tbody> <tr> <td>Last Name[field in Client Member General Layout]</td><td>LTRIM(RTRIM(RIGHT(#,20)))</td><td>Extract the last 20 chars of MemberName</td></tr> <tr> <td>Email ID(Field in Member General Layout)</td><td>Case len(Srcemail) when 0 then 'DEFAULT@DEFAULT.COM' Else Srcemail End</td><td>Default email id for records with blank email id</td></tr> </tbody> </table> CS_MAP_TRANSFORM table is setup with predefined transform values.			CS field that requires transformation	Transform Rule	Description	Last Name[field in Client Member General Layout]	LTRIM(RTRIM(RIGHT(#,20)))	Extract the last 20 chars of MemberName	Email ID(Field in Member General Layout)	Case len(Srcemail) when 0 then 'DEFAULT@DEFAULT.COM' Else Srcemail End	Default email id for records with blank email id
CS field that requires transformation	Transform Rule	Description									
Last Name[field in Client Member General Layout]	LTRIM(RTRIM(RIGHT(#,20)))	Extract the last 20 chars of MemberName									
Email ID(Field in Member General Layout)	Case len(Srcemail) when 0 then 'DEFAULT@DEFAULT.COM' Else Srcemail End	Default email id for records with blank email id									

Any errors through these process, process is stopped and error log is generated. UDA automatically converts the process to “Re-run” status.

5 Data Validation and Engine and Error DB Load

Data Validation at field level for mandatory fields, data type and data length and valid date formats begins after all the data transformation is completed. Minimal validation is performed to ensure load into ClaimSphere and then a detailed Data Cleansing ensues.

Below validations are performed:

- All mandatory fields required for respective ClaimSphere file types are validated. In case of missing mandatory fields, records are tagged with appropriate error description and dropped from further processing.
- All Date fields are validated for a valid date type, in case of any invalid date format/alphanumeric values, records are tagged with appropriate error description and dropped from further processing. Non Mandatory fields with empty data is accepted as NULL.
- All Integer fields are validated for a valid type, in case of any string format/alphanumeric values, records are tagged with appropriate error description and dropped from further processing. Non Mandatory fields with empty data is accepted as NULL.

	All Numeric/Decimal fields are validated for a valid integer type and data length, precision and scale, in case of any string format/alphanumeric values, records are tagged with appropriate error description and dropped from further processing. Transform rules are built to convert numeric fields to required CS format with accurate length and precision. Non Mandatory fields with empty data is accepted as NULL. Any character strings exceeding the required CS field length will be truncated and loaded. All error free records are loaded into CS_ENG_* tables for respective input file types and all error records are loaded into CS_ERR_ST_* tables with error information.
6	CS File Generation
	Once the data is loaded into the Engine tables, output files are generated in the CS Delimited text format files. - The files are generated with the same input source file name for easy identification and back tracking. - Control file metrics are captured with input count, output count and error count details at an individual file level. - Error details consolidated with client source file/table upload errors, processing errors and system error logs are generated as Error files for Client and application support team's review.

3. Getting Started – UDA

In UDA, setting up Source DB, Mapping DB, Rules DB (Lookup, Transformation, and Data Validation) varies from Client to client. This will be set up as one time activity during implementation phase.

ClaimSphere format DBs like CS_STG_*DB, CS_ENG_*, CS_ERR_ST_*, CS_File_Master DBs are static and do not vary/not required to customize unless there is change from ClaimSphere product end for a given input file version (example: version V12, Version V13 intake files).

3.1. Prerequisites

The following prerequisites needs to be fulfilled to proceed with configuration & deployment of UDA:

- The DB instance / server required to deploy ClaimSphere should be decided before setting up the UDA DB. The UDA would be set up on the same server if there is only one instance for the client or in one or 2 instances [to reduce traffic] in case of multi instance deployment of ClaimSphere application.
- Setting up pre-defined path to receive incoming Source data files.
- Setting up database connection with Client DB/DW along with execution rights & access for reading & writing files generated out of UDA.
- Environment setup & software requirements mentioned as part of Installation Section.
- Setting up pre-defined path to send CS format generated intake files to be consumed by ClaimSphere application.

- Setting up error and audit metrics communication between Client and UDA.
- Information on all the existing Source data files from client and details on the file format, data type.
- Technical Skills – SSMS, SSIS are required to configure and setup the UDA application.
- Knowledge on ClaimSphere HEDIS data intake Files, ClaimSphere application.

3.2. UDA Source Staging Table Setup

3.2.1. Source Mode as File(F) – Incumbent Vendor Layout

All existing incumbent vendor files, data type, file format and other information required to setup Source DB at UDA end is collected through the template shared to the POC from Client Business/IT team.

All Source File Layout and fields should be mentioned here.								
Brief Description about the file - The information in this file will be used to setup Source DB in Data Load Adapter application. Please do not leave any fields that is present in the files that will be sent.								
Source Filename	Source Column Name	Data Type	Length	Precision	Scale	IsNullable	Description	Sample Value
MemberBenefitsLayout1	HCID	Char	13			No	Unique member	HCN1234567890

With the information collected through the above template, create UDA source staging tables for each file layout in UDA Source DB. Current schema used throughout UDA application is DLA (Data Load Adaptor).

Table names starting with DLA.SRC_* with VARCHAR [255] as Data Type in order for the data to be loaded as such without any formatting.

Example

If below are the client file types:

S.No	FileType	File Key Word to identify
1	MemberBenefitsLayout1	MEM_ENR1
2	Enrollment	MEM_ENR2
3	MemberDemographics	MEM_DEM

Tables could be created with below names and data type as VARCHAR [255]:

- DLA.SRC_ENROLL_LAYOUT1
- DLA.SRC_ENROLL_LAYOUT2
- DLA.SRC_MEMBER_DEM

Any and All source files with a unique layout needs a table to be created, so that the data from the file will be loaded into the corresponding Source table.

3.2.2. Source Mode as Table(T) – Client DB/DW

All client tables identified with data required for CS file generation and column definition and relationship information required to setup Source DB at UDA end is collected through the template shared to the POC from Client Business/IT team.

<<screenshot>>

With the information collected through the above template, create UDA source staging tables for each file layout/client tables in UDA Source DB. Current schema used throughout UDA application is DLA (Data Load Adaptor).

Table names starting with DLA.SRC_* with VARCHAR [255] as Data Type in order for the data to be loaded as such without any formatting.

Example

If below are the client file types:

S.No	Client Tables	Stream Type
1	CLIENT_MemberBenefits_MASTER	Membership
2	CLIENT_MEMBERDEMO_MASTER	Membership
3	CLIENT_PROVIDER_DIRECTORY	Provider

Source staging Tables needs to be created with below names and data type as VARCHAR [255]:

- DLA.SRC_MEMBER_BENEFITS
- DLA.SRC_MEMBER_DEMO
- DLA.SRC_PROVIDER_DIRECTORY

Any and All tables identified with a unique layout needs a source staging table to be created, so that the a working copy of data from client database is pulled into the UDA tool to avoid contention, avoid Live data stream and better tracking. Only required columns & data from client table will be created within source staging tables. In case of select columns required, UDA has a provision to pull only select columns through configuration.

3.3. Source Staging DB –Configuration

Every client file/table will be identified with a Source file ID and Source Table ID inside UDA and the corresponding Source Table ID needs to be mapped / set up in below tables:

- SRC_FILE_MASTER
- SRC_LOAD_MASTER
- SRC_TABLE_MASTER

3.3.1. SRC_FILE_MASTER

Contains information about the File Type (any of the below and any additional layouts) and file key word to identify the unique layout. It contains the mapping between the file id and the table id to which the data needs to be loaded to.

SRC_FILEID	SRC_TABLEID	SRC_FILETYPE	SRC_FILE_KEYWORD	IsActive	ModifiedBy	ModifiedDate
1	1	ENROLLMENT	MEM_ENR	1	ADMIN	2020-12-14 06:56:46
2	2	MEM GENERAL	MEM_GNR	1	ADMIN	2020-12-14 06:56:46
3	3	PROVIDER	PRV_GNR	1	ADMIN	2020-12-14 06:56:46
4	4	PROVIDER	PRV_SPL	1	ADMIN	2020-12-14 06:56:46
5	5	PROVIDER	PRV_FACL	1	ADMIN	2020-12-14 06:56:46
6	7	PHARMACY	MEM_RX	1	ADMIN	2020-12-14 06:56:46
7	8	LAB	MEM_LAB	1	ADMIN	2020-12-14 06:56:46
8	9	VISIT	MEM_VISIT	1	ADMIN	2020-12-14 06:56:46

3.3.2. SRC_LOAD_MASTER

Contains information about the Client Table (any of the below and any additional layouts) and Source Table ID word to identify the unique layout. It contains the Loading stream classified as Membership, Provider, Visit, Lab and Pharmacy for parallel processing. It contains the mapping between the client table and corresponding source staging table to which the data needs to be loaded to.

SRC_TABLEID	SRC_TABLE_NAME	DLASRC_TABLE_NAME	SRC_FILETYPE	LOAD_STREAM	LOAD_STATUS
1	Client.MembershipEnroll	DLA.src_memberenroll	ENROLLMENT	1	Y
2	Client.MemberDemographic	DLA.src_membergeneral	MEM GENERAL	1	Y
3	Client.Provider	DLA.src_provider	PROVIDER	2	Y
4	Client.ProviderSpecialty	DLA.src_provider_specialty	PROVIDER	2	Y
5	Client.ProviderFacility	DLA.src_provider_facility	PROVIDER	2	Y
7	Client.PharmacyClaims	DLA.src_pharmacy	PHARMACY	3	Y
8	Client.LabClaims	DLA.src_lab	LAB	4	Y
9	Client.MedicalClaims	DLA.src_visit	VISIT	5	Y

3.3.3. SRC_TABLE_MASTER

Contains information about the Source column from each source DB (mapped to each client file/table from SRC_FILE_MASTER or SRC_LOAD_MASTER). This information will be used in the mapping of CS fields and Source fields.

	SRC_TABLEID	SRC_TABLE_NAME	SRC_COLID	SRC_COL_NAME	IsActive
1	1	DLA.src_memberenroll	1	ProductID	1
2	1	DLA.src_memberenroll	2	MemberID	1
3	1	DLA.src_memberenroll	3	EnrollmentDate	1
4	1	DLA.src_memberenroll	4	TerminationDate	1
5	1	DLA.src_memberenroll	5	BenefitID	1
6	1	DLA.src_memberenroll	6	ProviderID	1
7	1	DLA.src_memberenroll	7	GroupID	1
8	1	DLA.src_memberenroll	8	PayerID	1
9	1	DLA.src_memberenroll	9	Confidential	1

3.4. Client DB Extract Configuration:

Every Client table configured in the SRC_LOAD_MASTER need to have extract criteria configured within UDA. UDA should be able to extract only the membership and claims data for a particular Lookback period and for the Line of Business to be supported.

3.4.1. SRC_EXTRACT_RULES:

Minimal extraction criteria like Lookback period for membership, claims, LOB are already inbuilt and can be changed via configuration in SRC_EXTRACT_RULES table.

SRC_TABLEID	RULENAME	FIELDVALUE1	FIELDVALUE2	IsActive
1	LOOKBACK_MEMBERSHIP	3	6	1
9	LOOKBACK_MEMBERSHIP	3	6	1
9	LOOKBACK_CLAIMS	10	0	1
2	LOOKBACK_MEMBERSHIP	3	6	1
7	LOOKBACK_MEMBERSHIP	3	6	1
8	LOOKBACK_MEMBERSHIP	3	6	1
7	LOOKBACK_CLAIMS	10	0	1
8	LOOKBACK_CLAIMS	10	0	1

SRC_TABLEID – Corresponds to the client table id set up in SRC_LOAD_MASTER.

RULENAME – Refers to Lookback for membership/claims. This can be extended for LOB's supported (if it is a smaller list and a direct reference in the Client table) or can hold the Product /LOB cross-reference tables.

FIELDVALUE1 – Holds the no.of years.

FIELDVALUE2 – Holds the no.of months.

IsActive – Activate/Deactivate any of the rows. Only active row data is picked up for UDA processing.

Each of the client table that needs to be extracted with Lookback period need to have an individual entry in this table. This allows the tables to have individual lookback periods configured within a single run.

3.4.2. CLIENT_EXTRACT_CONFIG:

All other specific field level parameters can be configured in CLIENT_EXTRACT_CONFIGURATION table restricting UDA to pull only the required data from the client and aligning closely with ClaimSphere data Extraction guidelines.

Each client table identified by SRC_TABLEID drives the entire data extraction process from client DB/DW within UDA.

SRC_TABLEID	FIELDNAME	FIELDVALUE	OPERATOR	OPERAND	Parm_Condition_text	Parm_Seq	Parm_Inner_Seq	Use_Parm	Use_Field	Parm_Desc	IsActive
1	TERMINATIONDATE	LOOKBACK_ME...	>=	AND		2	1	Y	N	Apply lookback value to Member Enrollment date	1
1	TERMINATIONDATE	2018-01-01	>=	AND		2	1	Y	N	Apply lookback value to Member Enrollment date	0
1	JOIN				INNER JOIN DLACS_PRODUCT_REFERENCE ...	1	0	N	N	Join to Product Lookup table	1
1	ADDITIONAL_QUERY			AND	Client.MembershipEnroll CoverageIndicator = 'P'	3	1	N	N	Custom Query	1
2	JOIN				INNER JOIN Client.MembershipEnroll ON Client.M...	1	0	N	N	Join to Membership table	1
2	JOIN				INNER JOIN DLACS_PRODUCT_REFERENCE ...	1	0	N	N	Join to Product Lookup table	1
2	ADDITIONAL_QUERY			AND	Client.MembershipEnroll.TERMINATIONDATE >= ...	2	1	N	N	Custom Query	1
7	JOIN				INNER JOIN Client.MembershipEnroll ON Client.P...	1	0	N	N	Join to Membership table	1
7	JOIN				INNER JOIN DLACS_PRODUCT_REFERENCE ...	2	0	N	N	Join to Product Lookup table	1

FIELDNAME – refers to the column name in the client table. In the above screenshot, Terminationdate is the column of the table Src_tableid 1(to be referred with SRC_LOAD_MASTER).

FIELDVALUE – refers to the value to be compared with Fieldvalue/particular column in the client table. In the above screenshot, Terminationdate column of Src_tableid 1 is compared with values – “2018-01-01”. This field can hold:

Another column name / Lookback identifier (to be referenced with SRC_EXTRACT_RULES)

USE_FIELD – indicates if the FIELDVALUE is a column name or a data value to be compared.

OPERATOR – refers to the comparison operator. Valid values include (<,>,<=,>=,!,<>,IN,NOT IN,IS NULL,IS NOT NULL)

OPERAND – Only “AND” or “OR” values are allowed.

PARM_CONDITION_TEXT – refers to the additional query/ join conditions that the clients/ data extractors can write as a specific code while pulling the data.

All Join conditions & additional conditions that cannot be achieved through parametrization is simplified by allowing the client to enter a valid SQL code (only criteria).

fieldname	Parm_Condition_text
JOIN	INNER JOIN Client.MembershipEnroll ON Client.PharmacyClaims.MEMBERID = Client.MembershipEnroll.MEMBERID
JOIN	INNER JOIN DLACS_PRODUCT_REFERENCE ON Client.Pharmacyclaims.PRODUCTID = DLACS_PRODUCT_REFERENCE.PRODUCTID

A closer look at the above Join condition for a Pharmacy Client table indicates the Join condition with Enrollment database to extract only members part of membership and product type sent to Claimsphere.

“WHERE” condition or keyword is not to be used anywhere within the Parm_condition_text or Operand.

“Where” condition is formed with all other parameters programmatically considering multiple join conditions.

PARM_SEQ – refers to the order in which the filter needs to be applied. Always the Join conditions are placed at the first sequence, followed by other parameters.

PARM_INNER_SEQ – refers to the order in which the filter precedence needs to be applied within “AND 7 OR”. While placing OR conditions, extra precaution must be taken for the order to make sure the query results remain consistent. Inner sequence can help with OR conditions that need to be applied within an AND condition.

USE_PARM – indicates if the record is a Join/Additional query provided by client or a criteria to be dynamically formed using customer provided fieldvalues & fieldname.

N -> to be set when it is “JOIN” or “ADDITIONAL_QUERY”, where Parm_condition_text will be used as such.

Y -> to be set when FIELDNAME, FIELDVALUE, OPERAND, OPERATORS are used to frame the extract criteria.

Example extraction criteria for Pharmacy table client.pharmacyclaims

SRC_TA...	FIELDNAME	FIELDVALUE	OPERATOR	OPERAND	Parm_Condition_Text	Parm_Seq	Parm_Inner_Seq	Use_Parm	Parm_Desc	Use_Field
7	JOIN				INNER JOIN Client.MembershipEnroll ON Client.PharmacyClai...	1	0	N	Join to Membership table	N
7	JOIN				INNER JOIN DLA_CS_PRODUCT_REFERENCE ON Client.Ph...	2	0	N	Join to Product Lookup table	N
7	ADDITIONAL_QUERY			AND	Client.MembershipEnroll.TERMINATIONDATE >= LOOKBACK...	3	1	N	Custom Query	N
7	Prescribeddate	LOOKBACK_CLAIMS	>=	AND		4	1	Y	Apply lookback value to Prescribed Date	N
7	Denied	N	=	AND		5	2	Y	Only Paid & denied status	N
7	datasourcetype	PS,NS,SS	IN	AND		6	1	Y	Extract only appropriate datasource type	N

- 1) Join condition with client.membershipenroll table to make sure we are extracting only members part of membership pulled into Claimsphere, with Parm Seq as 1 indicating first record to be pulled for query formation, Use_Parm as N, indicating that Parm_condition_text is to be used and Use field as N since there is no parameterized query formation.
- 2) Join condition with Product_reference table to pull only MCE LOB with Parm Seq as 2 indicating second record to be pulled for query formation, Use_Parm as N, indicating that Parm_condition_text is to be used and Use field as N since there is no parameterized query formation.
- 3) Additional_Query for additional where clause, member termination date >= Lookback Membership value [will be substituted with '2017-06-01' as part of 3.5 yrs lookback guideline] with Parm Seq as 3 indicating third record to be pulled for query formation, Use_Parm as N, indicating that Parm_condition_text is to be used and Use field as N since there is no parameterized query formation.
- 4) Prescribeddate is to be compared with Lookback_Claims(to reference SRC_EXTRACT_RULES) using AND operand with Parm Seq as 4 indicating fourth record to be pulled for query formation, Use_Parm as Y, indicating to form the criteria using fieldvalue and operands and Use field as N since there is no column name used in comparison(lookback_claims is not column value, the value will be substituted from reference table).
- 5) Denied is to be compared with value 'N' using AND operand with Parm Seq as 5 indicating fifth record to be pulled for query formation, Use_Parm as Y, indicating to form the criteria using fieldvalue and operands and Use field as N since there is no column name used in comparison.
- 6) DataSourceetype is to be compared with value list 'PS,SS,NS' using AND operand with Parm Seq as 6 indicating sixth record to be pulled for query formation, Use_Parm as Y, indicating to form the criteria using fieldvalue and operands and Use field as N since there is no column name used in comparison.

Exact Query that is formed:

Inner join client.membershipenroll on clientpharmacyclaims.memberid =
client.membershipenroll.memberid

Inner join dla.cs_product_reference on clientpharmacyclaims.productid =
dla.cs_product_reference.productid

Where client.membershipenroll.terminationdate >= '2017-06-01'/* FROM SRC_EXTRACT_MASTER

And prescribeddate >= '2011-03-01'/* FROM SRC_EXTRACT_MASTER

And Denied = 'N'

And datasourcetype in ('PS','NS','SS')

3.5. CS File Master Configuration:

Every ClaimSphere file type required to be generated from UDA is configured in CS_FILE_MASTER table with details like – File Type, File Version, Staging Table Name, Engine Table Name and Error File Name.

Each File Type is given a **FILE_ID** which drives the entire data load, transformation, lookup and output file generation within UDA.

FILE_ID	CSTABLE_ID	CSFILE_NAME	CSTABLE_NAME	CS_STGTABLE_ID	CS_STGTABLE_NAME	CS_ERRTABLE_NAME	FILE_VERSION	IsActive
1	110	CS_MEMBERSHIP_ENROLL	DLACS_ENG_MEMBERENROLL	101	DLACS_STG_MEMBERENROLL	DLACS_ERR_ST_MEMBERENROLL	V12	1
2	111	CS_MEMBERSHIP_GENERAL	DLACS_ENG_MemberGENERAL	102	DLACS_STG_MEMBERGENERAL	DLACS_ERR_ST_MEMBERGENERAL	V12	1
3	112	CS_MEMBERSHIP_PCP	DLACS_ENG_MemberPCP	103	DLACS_STG_MEMBERPCP	DLACS_ERR_ST_MEMBERPCP	V12	1
4	113	CS_PROVIDER	DLACS_ENG_PROVIDER	104	DLACS_STG_PROVIDER	DLACS_ERR_ST_PROVIDER	V12	1
5	114	CS_PROVIDER_INFO	DLACS_ENG_PROVIDERINFO	105	DLACS_STG_PROVIDERINFO	DLACS_ERR_ST_PROVIDERINFO	V12	1
6	115	CS_MEMBER_VISIT	DLACS_ENG_VISIT	106	DLACS_STG_VISIT	DLACS_ERR_ST_VISIT	V12	1
7	116	CS_MEMBER_PHARMACY	DLACS_ENG_PHARMACY	107	DLACS_STG_PHARMACY	DLACS_ERR_ST_PHARMACY	V12	1
8	117	CS_MEMBER_LAB	DLACS_ENG_LAB	108	DLACS_STG_LAB	DLACS_ERR_ST_LAB	V12	1
9	118	CS_MEMBER_SUPPLEMENTAL	DLACS_ENG_SUPPL	109	DLACS_STG_SUPPL	DLACS_ERR_ST_SUPPL	V12	0
10	210	CS_MEMBERSHIP_ENROLL	DLACS_ENG_MEMBERENROLL_V13	200	DLACS_STG_MEMBERENROLL_V13	DLACS_ERR_ST_MEMBERENROLL_V13	V13	1
11	211	CS_MEMBERSHIP_GENERAL	DLACS_ENG_MEMBERGENERAL_V13	201	DLACS_STG_MEMBERGENERAL_V13	DLACS_ERR_ST_MEMBERGENERAL_V13	V13	1
12	212	CS_MEMBERSHIP_PCP	DLACS_ENG_MEMBERPCP_V13	202	DLACS_STG_MEMBERPCP_V13	DLACS_ERR_ST_MEMBERPCP_V13	V13	1
13	213	CS_PROVIDER	DLACS_ENG_PROVIDER_V13	203	DLACS_STG_PROVIDER_V13	DLACS_ERR_ST_PROVIDER_V13	V13	1
14	214	CS_PROVIDER_INFO	DLACS_ENG_PROVIDERINFO_V13	204	DLACS_STG_PROVIDERINFO_V13	DLACS_ERR_ST_PROVIDERINFO_V13	V13	1

Some of the CS intake files are optional for some clients and for some they are not.

Example – Member Delivery, Supplemental, Visit Encounter, Procedure, Diagnosis, Pharmacy C, LISHIST files are optional files.

Based on the client, place an entry in the CS_File_Master table and use the auto-generated file id within UDA.

3.6. CS Table Configuration:

For a given version (V12/V13 or any future versions), CS_Stg table and CS_Eng table and any other reference tables [example product id, product type mapping] are created in the ClaimSphere Staging Table formats and appended with version number (example: Version V13 – all stg, eng and error tables appended with _V13 and _V14 for future V14 versions).

Equivalent format of below tables (only for the CS file layouts generated from UDA) are created in UDA CS DB.

ST_MembershipGeneral	ST_MembershipEnroll
ST_Monthly_Membership_Detail	ST_MemberPCP
ST_Member_Delivery	ST_Visit
ST_Pharmacy	ST_Lab
ST_Provider	ST_Provider_Info
ST_Supplemental	ST_visit_encounter
ST_visit_diagnosis	ST_visit_procedure
ST_pharmacy_Clinical	ST_lis_history

The table scripts for these tables are already available within UDA application and need to be set up in the server chosen for implementation.

Staging Tables	Engine Tables	Error Tables
• CS_STG_MEMBERGENERAL • CS_STG_MEMBERENROLL • CS_STG_MEMBERPCP • CS_STG_PROVIDER • CS_STG_PROVIDERINFO • CS_STG_VISIT • CS_STG_PHARMACY • CS_STG_LAB	• CS_ENG_MEMBERGENERAL • CS_ENG_MEMBERENROLL • CS_ENG_MEMBERPCP • CS_ENG_PROVIDER • CS_ENG_PROVIDERINFO • CS_ENG_VISIT • CS_ENG_PHARMACY • CS_ENG_LAB	• CS_ERR_ST_MEMBERGENERAL • CS_ERR_ST_MEMBERENROLL • CS_ERR_ST_MEMBERPCP • CS_ERR_ST_PROVIDER • CS_ERR_ST_PROVIDERINFO • CS_ERR_ST_VISIT • CS_ERR_ST_PHARMACY • CS_ERR_ST_LAB

3.7. CS Table Master Configuration:

CS_STG_TABLE_MASTER contains all CS Staging Tables and Reference tables with details like – Table ID, Column ID, Column Name, Data Type[int,char,varchar,numeric or bit, etc.] and length[data length],precision and scale for numeric and decimal data types,IsNullable as Y or N are set up.

The details like:

- 1) Data type, length, isnullable, precision and scale are used in Field level validation. Only records satisfying the field attributes will be error free, others will be tagged with error and stopped from further processing.
- 2) Code ID – The column is tagged with the Lookup Id from the table – CS_Code_Lookup to indicate that this column in ClaimSphere table requires translation to CS format as set up in CS_Code_Lookup table before the output file is generated.
- 3) Transform ID - The column is tagged with the Transform Id from the table – CS_Map_Transform to indicate that this column in ClaimSphere table requires transformation to CS format as set up in CS_Map_Transform table before the output file is generated.
- 4) IsActive – This indicates that the column in the CS Stg/Eng table is active/exist in the layout to be generated. Any column that is marked as inactive will be omitted from the output layout and mapping.
- 5) Col Seq Number – This number indicates the order of fields to be generated in Output CS file, data insertion into ENG table, STG table and Err tables. Any incorrect order will result in failure of mapping, data load and package.
- 6) HEDIS, STARSERV, COHORT, CLINICAL, ALL_REQ – These fields indicate the mandatory/required status of each column in the corresponding ClaimSphere applications.
Example - Starserv column corresponds to Startserv application in ClaimSphere Suite and a value 1 represents mandatory and 0 represents non-mandatory/optional field. These fields are used to drive the Warning message in case of absence of the mandatory fields based on the value in the table master.

The CSTable ID and CSCOLId generated in this table will drive the Field level mapping, Transformation, Lookup and Output file generation. Any inconsistency within the UDA CS DB with respect to table ID, Table name and Col ID and Col Name will cause failure of mapping, data load and package.

Each Intake File Type is given a CS File ID which drives the entire data load, transformation, lookup and output file generation within UDA.

CSCOL_ID	CSTABLE_ID	CSCOL_NAME	CSTABLE_NAME	CSDATA_TYPE	CSLENGTH	CSPRECISION	Isnullable	CODE_ID	CSCOL_SEQ_NO	TRANSFORM_ID	CSSCALE	HEDIS	COHORT	STARSRV	CLINICAL	ALL_REQ
1	101	Row_ID	DLACS_STG_MemberEnroll	int	4	10	0	0	1	0	NULL	1	1	1	1	1
2	101	Member_ID	DLACS_STG_MemberEnroll	UDT_ID	100	0	0	0	2	0	NULL	1	1	1	1	1
3	101	Plan_Member_ID	DLACS_STG_MemberEnroll	UDT_ID	100	0	1	0	3	0	NULL	0	1	1	0	0
4	101	Subscriber_ID	DLACS_STG_MemberEnroll	UDT_ID	100	0	1	0	4	0	NULL	0	0	0	0	0
5	101	Start_Date	DLACS_STG_MemberEnroll	date	3	10	0	0	5	0	NULL	1	1	1	1	1
6	101	End_Date	DLACS_STG_MemberEnroll	date	3	10	0	0	6	152	NULL	1	1	1	1	1
7	101	Contract_ID	DLACS_STG_MemberEnroll	varchar	16	0	1	0	7	0	NULL	1	1	0	0	0

Error and Engine table details can be inserted as well (Non Mandatory).

Data Type, Length should reflect the data type and length of HDS.ST_* tables.

Example – In the above sample screenshot, for the Membership General Layout, Staging Table ID is 101 and all columns part of CS STG table is listed with the correct sequence number, data type, length, precision and scale similar to that of HDS.ST_MEMBERSHIPGENERAL.

	TRANSFORM_ID	TRN_ID	TRANSFORM_VALUE	VALUE_DESC
1	200	1	SUBSTRING(trim(trim(#)),1,14)	Useful for Taking substring of first 14 chars from ...
2	201	2	SUBSTRING(trim(trim(#)),15,2)	Useful for Taking substring of 2 chars from Cs Co...
3	150	3	(Case ISNULL(LEN(#),0) When 0 then 'DEFAULT@DEFAULT.C...	Default Email ID
4	152	4	(Case # When '12/31/2199' then '12/31/9999' When " then '12/...	Default Term ID

We can also see code id, Transform ID marked to reflect the changes:

	CODE_ID	CODE_LKUP_ID	CSCODE_VALUE	CODE_DESC	SRC_VALUE
	101	5	Y	Primary Coverage Indicator	P
	101	6	N	Secondary Coverage Indicator	S
	101	7		No Coverage Indicator	
	50	8	11	Ethnicity-Hispanic or Latino	Y
	50	9	12	Ethnicity- Not Hispanic or Latino	N
	50	10	18	Ethnicity- Declined	D
	50	11	19	Ethnicity- Unknown	
	48	12	09	Race - Unknown	Unknown
	48	13	08	Race - Declined	Declined
0	48	14	07	Race - Two or more	Two or more races
1	48	15	06	Race - Some other	Some other races
2	48	16	05	Race - Native Hawaiian and Other Pacific Islander	Native Hawaiian and Other Pacific Islander
3	48	17	04	Race - Asian	Asian

3.8. Client Layout – CS Layout Mapping Configuration:

Once the Source Staging tables and CS Stg & Eng tables are setup within UDA and Master tables are loaded with information, mapping between Source and CS DB needs to be achieved. This process involves the participation from:

- Client Business / Technical team who is knowledgeable on the existing source side files.
- Technical CS consultant or SME who is driving discussions on data mapping from ClaimSphere team who is knowledgeable in HEDIS Input files and ClaimSphere Tool.

Below tables will be set up at the end of the Field Level Mapping:

- CS_STG_Proj_Layout
- CS_STG_Proj_Mapping

The below template will be shared with Client team to map the source side field with the CS fields. The template is self-explanatory/walk-through can be scheduled to explain the CS Side Layout fields, data type, length and description of what these fields mean in HEDIS CS tool.

Member General Layout - Mapping between Client and Claimsphere												
No. Of Source files used to arrive at the columns		Brief Description about the file - Mandatory File for Data refresh One record per Member & Product combination										
Source file names used (please concatenate using semicolon - ; in the order)		MemberBenefitsLayout1; MemberDemoLayout1; MemberDemoLayout2										
Key fields to link between source files		Source file 1 & 2	HCID, PlanNumber									
		Source file 1 & 3	HCID									
		Source file 2 & 3	HCID, SubscriberID									
Claimsphere Header Name	Data Type	Length	Required for HEDIS	Expected Value	Is Nullable	Description	Sample Value	Source file Name	Source Column Name	Data Type	Length	Precision(Abs Numeric, Dec)
Member_ID	char	100	Required		No	Unique member ID that is referenced across Membership Enrollment, Visit, Pharmacy, Lab Observation, Member's PCP, Member Delivery and Supplemental files Blank/Null records will be rejected and reported in error log		MemberBenefitsLayout1	HCID	Char	13	
ast_Name	char	60	Required		No	Member Last Name (e.g. for the Member Name John D Paul, Last Name would be Paul) Blank/Null records will be rejected and reported in error log		MemberDemoLayout1	MemberName	Char	60	

- The client is required to go over the CS side fields and map the appropriate client side file/table and its column name.
- Every CS intake file layout is provided as a single sheet within the workbook with pre-loaded values of fields, description, datatype, length, sample values and expected values.
- Excel template is attached in Annexure.
- Once the mapping configuration document is approved from the representatives of Client and ClaimSphere team, technical contact from ClaimSphere team can setup the tables for Layout level mapping and Field level mapping for a particular file type.

3.8.1. CS_STG_Proj_Layout

The CS_STG_PROJ_LAYOUT table contains the details like:

- **FileID** – ClaimSphere File Type, Maps to FileID in CS_FILE_MASTER
- **LayoutID** – Auto generated ID to identify the layout in CS_STG_PROJ_MAPPING table
- **No. Of Template** – Denotes the number of source file layouts that is required to form the particular ClaimSphere layout. This is derived from the excel template as shown below
- **Reference ID** – If there are more than one layout involved in forming a CS file type, nature of join condition between the source files and the join key is captured in CS_MAP_REFERENCE table.
 - Note – For every additional layout, there will be multiple rows in the CS_STG_Proj_Layout table with a new Referential ID field. More explanation on the CS_MAP_Transform is handled later in this section.
 - The Referential ID from the table is populated here to facilitate dynamic mapping between the Source DB tables.
- **Condition ID** – If there is an additional filter condition that needs to be provided while joining multiple source layouts or even while extracting data from a single layout, filter condition can be mapped as free text in the CS_MAP_CONDITION table and the condition ID is populated in this table.
 - If there are more than one condition ID mapped for forming the layout, all of the conditions will be concatenate using “AND” operator.
 - Any operation involving “OR” operator needs to be handled within a single condition id since it is a free flowing text, that will be concatenated as such.
- **IsActive** – If the Layout mapping requires to be considered or not.
- **Combined_FileID** and **IsCombined_Layout** is for extended use.

A sample project layout configuration is given as below. Taking **Example** of File ID = 1[from CS_FILE_MASTER,], CS file name as CS_MEMBERSHIP_ENROLL.

To form CS_Membership_Enroll file, **3 layouts** are used and there **are 2 referential ID – 1, 2** that explains the table join details among the 3 layouts and **2 additional where clause text with condition id 1 and 2** are configured.

	Layout_ID	File_ID	No_of_Template	Combined_FILEID	REFERENTIAL_ID	IsCombined_Layout	CONDITION_ID	IsActive
1	1	1	3	0	1	0	1	1
2	2	1	3	0	2	0	2	1
3	3	2	1	0	0	0	0	1
4	4	3	1	1	0	1	0	1
5	5	4	1	0	0	0	0	1
6	6	5	4	0	450	0	3	1
7	7	5	4	0	451	0	0	1
8	8	5	4	0	452	0	0	1

CS_MAP_Reference

The CS_Map_Transform table contains the join conditions, join type and join key between 2 tables. This is used in CS_STG_PROJ_LAYOUT.

- **Referential ID** – Links the join condition for a CS file type in CS_STG_PROJ_LAYOUT
- **First Table Name** – Name of the first SRC_* table name
- **Second Table Name** – Denotes the second SRC_* table / layout that is involved in the CS File ID formation.
- **First Map Key** – Key column name on the First Source table that is to be joined with the Second Table column.
- **Second Map Key** – Key column name on the Second Source table that is to be joined with the First Table column. If there are more than one join keys on which the 2 tables are mapped, we need to repeat the same Referential ID, first table name, second table name.
- **Join Type** – Denotes the type of join – INNER, LEFT, RIGHT, FULL, OUTER. This needs a careful understanding of source side data and link between the 2 layouts. This should be clarified before the approval of Mapping Configuration document between Client and CS team.
- **Additional Columns** – Reserved for extended / future use.

MAP_REFERENCE_ID	REFERENTIAL_ID	First_SRC_Table	Second_SRC_Table	First_Map_Key	Second_Map_Key	Join_Type	Additional_Col1	Additional_Col2	IsActive
1	1	DLA.src_memberenroll	DLA.SRC_MEMBERGENERAL	MemberID	MemberID	Inner	A	B	1
2	1	DLA.src_memberenroll	DLA.SRC_MEMBERGENERAL	ProductID	ProductID	Inner	A	B	1
3	2	DLA.src_memberenroll	DLA.CS_PRODUCT_REFERENCE	ProductID	ProductID	Inner			1
4	450	DLA.SRC_Provider	DLA.SRC_PROVIDER_Specialty	ProviderID	ProviderID	Inner			1
5	451	DLA.SRC_Provider	DLA.SRC_PROVIDER_Facility	ProviderID	ProviderID	LEFT			1
6	452	DLA.SRC_Provider	DLA.SRC_PROVIDER_Specialty_PVT	ProviderID	ProviderID	INNER			1
7	453	DLA.SRC_PHARMACY	DLA.CS_PRODUCT_REFERENCE	ProductID	ProductID	INNER			1

Example – To populate the table based on Mapping Configuration Document.

Member General Layout - Mapping between									
Brief Description about the file - Mandatory File for Data refresh One record per Member & Product combination									
No. Of Source files used to arrive at the columns						3			
Source file names used (please concatenate using semicolon - ; in the order)						MemberBenefitsLayout1; MemberDemoLayout1; MemberDemoLayout2			
Key fields to link between source files						HCID, PlanNumber			
						HCID			
						HCID, SubscriberID			

In the above snippet, there are 3 layouts required to map the CS file type - Member General.

The 3 tables used are

- 1) MemberBenefitsLayout1
- 2) MemberDemoLayout1
- 3) MemberDemoLayout2

- The Join Condition between the files are determined as INNER for all 3 layouts. Only Matching records are to be extracted from all 3 layouts.
- Join keys between First and Second Layout is marked as HCID & Plan Number in MemberBenefitsLayout1 and MemberDemoLayout1
- Join Keys between First and Third Layout is HCID
- Join Keys between Second and Third Layout is HCID and Subscriber ID.

Based on the above scenario, the below mapping can be done in CS_MAP_REFERENCE table.

First SRC Table	Second SRC Table	First Map Key	Second Map Key	Join Type	IsActive	Referential ID
MemberBenefitsLayout1	MemberDemoLayout1	HCID	HCID	Inner	1	100
MemberBenefitsLayout1	MemberDemoLayout1	PlanNum	PlanNum	Inner	1	100
MemberBenefitsLayout1	MemberDemoLayout2	HCID	HCID	Inner	0	101
MemberDemoLayout1	MemberDemoLayout2	HCID	HCID	Inner	1	102
MemberDemoLayout1	MemberDemoLayout2	SubscriberID	SubscriberID	Inner	1	102

CS_MAP_CONDITION

The CS_Map_Condition table contains additional filter condition that needs to be provided while joining multiple source layouts or even while extracting data from a single layout, filter condition can be mapped as free text in the CS_MAP_CONDITION table and the condition ID is populated in this table.

- **Condition ID** – Links the condition ID for a CS file type in CS_STG_PROJ_LAYOUT
- **Condition Text** – Free flowing Text that contains the filter conditions – Where, Group By or Order by Clause to be provided at a layout level.
- **Condition Type** – Denotes the Table level or Column level mappings. Currently only Table level conditions are handled.
- **Additional Columns** – Reserved for extended / future use.

Example Condition Text that can be used.

CONDITION_ID	CONDITION_TEXT	CONDITION_TYPE	Additional_Col1	IsActive
1	TerminationDate = (SELECT TOP 1 MAX(C.TerminationDate) F...	TABLE		1
2	DLA.CS_PRODUCT_REFERENCE.ISACTIVE = 1	TABLE		1
3	DLA.SRC_PROVIDER.PROVIDERID <> 'UNKNOWN'	TABLE		1
4	(DLA.SRC_LAB.MEDICALCLASS = 'L' OR (DLA.SRC_LAB.ME...	TABLE		1
5	(DLA.SRC_LAB.LOINCCODE IS NOT NULL OR DLA.SRC_LA...	TABLE		1
6	(DLA.SRC_VISIT.MEDICALCLASS IN ('M','F'))	TABLE		1

Points to Remember:

- The Condition_Text will require caution as any syntax issue can cause the package to fail.
- Do not include “WHERE” or “AND” at the start of Condition_Text as the application will concatenate multiple condition_text with “AND” operator.
- Caution to be exercised while providing “OR” operators with proper open and close braces as these will impact the result set.
- It is discouraged to use GROUP BY as the source columns are pulled through dynamic query formation.
- In case of ORDER BY, it is encouraged to use specific column name from source table.

This completes the Layout Level mapping for different Source Layouts.

3.8.2. CS_STG_PROJ_MAPPING

Once a particular CS Layout is formed, field level mapping between the selected source tables and CS STG table needs to be mapped, so that the fields will be extracted as per mapping and join conditions and additional filter conditions will be applied from CS_STG_PROJ_LAYOUT.

The CS_STG_PROJ_MAPPING table contains the details like:

- **FileID** – ClaimSphere File Type, Maps to FileID in CS_FILE_MASTER
- **LayoutID** – Layout ID to identify the layout in CS_STG_PROJ_LAYOUT table
- **CSTBL_ID** – Denotes the CS Staging Table ID that needs to be loaded with data from the source for the particular FileID. The table reference should be the same tableid, column id, column name from CS_STG_TABLE_MASTER table.
- **CSCOL_ID** – Denotes the CS Staging Column ID that needs to be mapped with data from the source column for the particular FileID. The table reference should be the same tableid, column id, column name from CS_STG_TABLE_MASTER table.
- **CSCOL_NAME** – Denotes the CS Staging Column Name that needs to be mapped with data from the source column for the particular FileID. The table reference should be the same tableid, column id, column name from CS_STG_TABLE_MASTER table.
- **SRCTBL_ID** – Denotes the Source Table ID that needs to be pulled as the source table for the particular FileID. The table reference should be the same tableid, column id, column name from SRC_TABLE_MASTER table.
- **SRCTABLE_NAME** – Denotes the Source Table Name that needs to be pulled as the source table for the particular FileID. The table reference should be the same tableid, column id, column name from SRC_TABLE_MASTER table.
- **SRCCOL_ID** – Denotes the Source Column ID that needs to be pulled as the source column from source table [SRCTABLE_NAME] for the particular FileID. The table reference should be the same tableid, column id, column name from SRC_TABLE_MASTER table.
- **SRCCOL_NAME** – Denotes the Source Column Name that needs to be pulled as the source column from source table [SRCTABLE_NAME] for the particular FileID. The table reference should be the same tableid, column id, column name from SRC_TABLE_MASTER table.
- **DEFAULT_VALUE** – If there is no mapping from Source table / Source column for a particular CSColumn, a default value can be mentioned, so that instead of sourcetable.source column, default value will be pulled into CS_STG table.
 - Note – Only one default value can be mentioned.
 - If the default value needs to be changed according to the presence of value in source column, use CS_MAP_Transform and map the TRANSFORM_ID in CS_STG_TABLE_MASTER and not in this table.
- **Transform ID** – If there is any criteria to map more than one source column to a CSColumn, use this Transform ID column and set up the Transform value in CS_MAP_TRANSFORM table.
 - The Transform ID here will only be used for multiple source fields to a single CS column.
Example If CSColumn has Provider Address as part of CS_PROVIDERINFO layout and Source system stores the Address Line 1 and Address Line 2 as 2 separate fields, a concatenation of 2 source fields can be written as a Transform and used here.
 - Any other transform needs to be addressed only through CS_STG_TABLE_MASTER.
- **IsActive** – If the Layout mapping requires to be considered or not.

A sample project mapping configuration is given as below. Taking **Example** of File ID = 1[from CS_FILE_MASTER,], CS file name as CS_MEMBERSHIP_ENROLL and CSTBL_ID as 101[CS_FILE_MASTER, CS_STG_TABLE_MASTER all reflect the same value].

To form CS_Membership_Enroll file, each field in CSTbl_ID is mapped to a field from source table and a source column as below.

In case of no source values available, a default value is mentioned.

Map_ID	Layout_ID	File_ID	CSTBL_ID	CSCOL_ID	CSCOL_NAME	SRCTBL_ID	SRCTBL_NAME	SRCCOL_ID	SRCCOL_NAME	DEFAULT_VALUE	TRANSFORM_ID	IsActive
1	1	1	101	2	Member_ID	1	DLA.src_memberenroll	2	MemberID		0	1
2	1	1	101	3	Plan_Member_ID	1	DLA.src_memberenroll	2	MemberID		0	1
3	1	1	101	4	Subscriber_ID	1	DLA.src_memberenroll	27	SubscriberID		0	1
4	1	1	101	5	Start_Date	1	DLA.src_memberenroll	3	EnrollmentDate		0	1
5	1	1	101	6	End_Date	1	DLA.src_memberenroll	4	TerminationDate	12/31/9999	0	1
6	1	1	101	7	Contract_ID	1	DLA.src_memberenroll	5	BenefitID		0	1
7	1	1	101	8	Plan_ID	1	DLA.src_memberenroll	31	CMSPlanID		0	1
8	1	1	101	9	Product_ID	1	DLA.src_memberenroll	1	ProductID		0	1
9	1	1	101	10	PCP_ID	1	DLA.src_memberenroll	6	ProviderID		0	1
11	1	1	101	12	Employee_Flag	0		0		0	0	1
12	1	1	101	13	LIS_Flag	0		0		0	0	1
13	1	1	101	14	Primary_Payer	1	DLA.src_memberenroll	50	CoverageIndicator		0	1
14	1	1	101	15	Dental_Flag	1	DLA.src_memberenroll	24	DentalBenefit	N	0	1
15	1	1	101	16	Drug_Flag	1	DLA.src_memberenroll	14	PharmacyBenefit	N	0	1

Points to Remember:

- The SRCTable name and Src Col name should be carefully mapped and should be in coherence with SRC_TABLE_MASTER.
- The Transform ID value will require caution as any syntax issue will cause the package to fail.
- If there is Default Value present, with Source Table name and Source Column name as non-blank, default value will not be applied. Example, Default values of Dental_Flag and Drug_Flag and End_Date will not be applied.
- Internal reference tables can also be used as Source. **Example** – A condition check to make sure to pull records with Product ID setup & active in Product Cross Reference tables. In this case, Src Table ID can be 0, but SRCTable name and col name should be marked with valid values and an appropriate join condition and join keys should be mapped in CS_MAP_REFERENCE table.

3.9. Output File Configuration

The CS_OUTFILE_CONFIGURATION table contains the below details to be set up to generate CS formatted data loader files.

CS_FILEID – denotes the File_ID in CS_FILE_MASTER table.

IsExactFileName – If set to 1, the CS format output files of UDA will be generated with original source file name. If set to 0, the CS format output files of UDA will use Outfile_NM_Constant to create the files.

Outfile_NM_Constant – To be used along with source file name / stand-alone file name for the output Cs generated file from UDA.

IsTimestamp_Append – Configuration to append timestamp to the source input file name if set up as “1”.

Example – Different file names used for various CS_FileID/CS generated output file types.

	CS_Outfile_ConfigId	CS_FILEID	IsExactFileName	IsTimestamp_Append	Outfile_NM_Constant	Add_Col1	Add_Col2	IsActive
1	1	1	1	1	ENROLLMENT			1
2	2	2	1	1	MEM_GENERAL			1
3	3	3	1	1	PROVIDER			1
4	4	4	1	1	PROVIDER			1
5	5	5	1	1	PROVIDER			1
6	6	6	1	1	PHARMACY			1
7	7	7	1	1	LAB			1
8	8	8	1	1	VISIT			1

3.9.1. Custom Procedure Configuration

If there are any specific preprocessing or post processing is required as part of customization for a client, whose layout could not be generalized/requires specific custom procedures to be written, UDA provides Customization package to run specific procedures for certain layouts.

The below table - CS_CUSTOM_PROCEDURE can be used to execute the SPs in the particular process order desired. The process can be customized to be executed at any part of the UDA Workflow.

Example – The Below 2 process can be run after the source file upload and before the file mapping starts, so as the mapping process consumes the source table setup by the 2 Custom SPs.

Process_ID	PROCESS_NAME	Process_Order	IsActive
1	EXEC DLA.CSCustomProviderSpecialty	1	1
2	EXEC DLA.CSCUSTOMPROVIDERNPILOAD	2	1

3.10. Package Configuration

All the below package list will be configured in the server where UDA will be deployed.

- Each source file upload package will have the Source File Path, Output FilePath, Archive File path and error File path configured along with the layout identification keyword.
- UDA_PackageList table contains the All File path - Source FilePath, Output FilePath, Archive File path and the process Status of each Package during a cycle run.
- Any package can be de-activated using the IsActive flag.

SrcLoad_Mode	AuditPackageListId	PackageName	Search_text	FileType	SourceFilePath	DestinationFilePath
	1	DLA_Master.dtsx	*File_List*.txt	NA	\\10.28.161.200\DLA\InputFiles\	\\10.28.161.200\DLA\OutputFiles\
F	2	DLA_SRC_MembershipGeneral.dtsx	*MEM_GNR*.txt	NA	\\10.28.161.200\DLA\InputFiles\	\\10.28.161.200\DLA\OutputFiles\
F	3	DLA_SRC_MembershipEnroll.dtsx	*MEM_ENR*.txt	NA	\\10.28.161.200\DLA\InputFiles\	\\10.28.161.200\DLA\OutputFiles\
F	4	DLA_SRC_Provider.dtsx	*PRV_GNR*.txt	NA	\\10.28.161.200\DLA\InputFiles\	\\10.28.161.200\DLA\OutputFiles\
F	5	DLA_SRC_Provider_Specialty.dtsx	*PRV_SPL*.txt	NA	\\10.28.161.200\DLA\InputFiles\	\\10.28.161.200\DLA\OutputFiles\
F	6	DLA_SRC_Provider_Facility.dtsx	*PRV_FACL*.txt	NA	\\10.28.161.200\DLA\InputFiles\	\\10.28.161.200\DLA\OutputFiles\
F	7	DLA_SRC_Pharmacy.dtsx	*MEM_RX*.txt	NA	\\10.28.161.200\DLA\InputFiles\	\\10.28.161.200\DLA\OutputFiles\
F	8	DLA_SRC_Lab.dtsx	*MEM_LAB*.txt	NA	\\10.28.161.200\DLA\InputFiles\	\\10.28.161.200\DLA\OutputFiles\
F	9	DLA_SRC_Visit.dtsx	*MEM_VISIT*.txt	NA	\\10.28.161.200\DLA\InputFiles\	\\10.28.161.200\DLA\OutputFiles\
	10	DLA_DST_MembershipGeneral.dtsx	NA	CS_MEMBERSHIP_GENERAL	\\10.28.161.200\DLA\InputFiles\	\\10.28.161.200\DLA\OutputFiles\
	11	DLA_DST_MembershipEnroll.dtsx	NA	CS_MEMBERSHIP_ENROLL	\\10.28.161.200\DLA\InputFiles\	\\10.28.161.200\DLA\OutputFiles\
	12	DLA_DST_MemberPCP.dtsx	NA	CS_MEMBERSHIP_PCP	\\10.28.161.200\DLA\InputFiles\	\\10.28.161.200\DLA\OutputFiles\

3.10.1. Process Status & Rerun of Failed Packages

Each UDA package can have 4 process status:

- **Yet to Start** – denotes the ‘to be started’ status of the package when the UDA application is started.
- **Failed** – denotes that the package has failed in the current/last execution
- **InProgress** – denotes that the package is currently under execution.
- **Completed** – denotes that the package has successfully completed the current/last execution.

Once the process has started, status of each individual package can be viewed from the table – DLA .DLA _PACKAGELIST table. (Keep refreshing the query in SQL Window, packages will go through status – Yet to start -> In Progress -> Completed).

In case of a Package failure and a rerun after correction is required, UDA controls the rerun with the below table - DLA _ETL_CONFIG

Load Type Flag – F - Indicates Full Run; R – Indicates Rerun mode, where failed packages can be rerun.

Source FileLoading Flag – Y will run all the Source file/table to Source DB Load. If set N, will skip the Source DB Load.

Generate CS Files Flag – Y will run the mapping, lookup, transformation, validation and CS format file generation. If set N, will skip the entire process.

Generate CTRL and ERR Files Flag – Y will run the generation of Control file and Error files for the complete run. If set N, will skip the entire process.

Applications_setup –Applications from ClaimSphere Suite, which are opted in by the client. Multiple values are configured with semicolon. Here in below example, HEDIS, COHORT and STARSERV are the applications that the client has opted to avail services from Cognizant.

Version_to_run – Client intake files can be converted into corresponding intake file version used in ClaimSphere. It can be Version 12/Version 13 and any other future versions depending on NCQA revisions every year. The version mentioned here will be used to drive the mapping and application output file generation.

Reporting Year – End of year values like 2020, 2021, 2022 is used to derive the Claims Count Report generated by UDA.

ETL_Config_ID	Conf_Key	Conf_Value	Schema_Name	Description
1	LoadType_Flag	F	DLA	LoadType Flag, either (F) or (R)
2	Source_FileLoading_Flag	Y	DLA	SRC_FileLoading Flag, either (Y) or (N)
3	Generate_CS_Files_Flag	Y	DLA	Generate CS Files Flag, either (Y) or (N)
4	Generate_CTRL_and_ERR_Files_Flag	Y	DLA	Generate Control,Error Files Flag, either (Y) or (N)
7	Applications_setup	HEDIS;COHORT;STARSERV	DLA	Provide all application names that this DLA will be...
8	Version_to_run	V13	DLA	Set only one version to run.Values should be eith...
9	ReportingYear	2020	DLA	Set MeasurementYear
11	SOURCE_LOAD_MODE	T	DLA	Set Source as Table - T OR File - F

Steps to be followed for rerun:

If any package fails, the Process Load Type – automatically sets to “R” and status of the Package – remains “Failed”.

If there are packages running in parallel, only the failed packages stops the execution, all other parallel packages[that is in the same workflow with no dependency] completes the execution and status is updated to “Completed”.

The next process / dependent process of failed package, stops execution and error log is written with failure details.

Failed package can be part of any of the following workflow:

- **Source Data File Load** – If any of the source data file / table load package fails, other parallel source data file load succeeds. The entire execution stops with Source Data Load and will not proceed to Data mapping and creation of CS Gen files.
- **CS Data Load and Outfile Generation** – If any of the processing package fails, only the corresponding package stops execution and remains in 'Failed' status. Rest of the load and outfile generation packages continue their process and stops at the end of Output file Generation. The flow will not proceed to Control files, Audit files and Error file generation.

In order to rerun any specific packages or load only certain type of files:

Example – Customers have sent additional Visit and Pharmacy files post the complete full run,

- Depending on which package has failed, Source File Load flag and Generate CS File Flag can be set to Y or N and process can be kick started.
- If the process is in “R” mode, all “Completed” packages will be skipped and “Failed” and “Yet to start” packages will be re-started.
- If any of the “Completed” status process/package needs to be rerun, LoadType should be “R” mode and manually update Processstatus to “Yet to start” before rerunning the process.
- If the process is in “F” or “R” mode and any of the packages are in ‘In progress’ status, it will be considered as duplicate run and the process will error out. This is to prevent multiple users triggering the UDA process simultaneously.

3.10.2. Data Cleanup & Archival:

- The first step in package execution is “Initializing the UDA source and CS DB” and archiving Source Control, CS Control file and Error Summary for Audit purpose.
- All the SRC_* tables, CS_STG*, CS-ENG*, CS_ERR_ST* tables should be part of cleanup along with Control, Audit tables and few process tables.
- The initialization process will not run if the Load Flag is R or if the Load Flag is “F” and status of any packages = ‘InProgress’. This is to prevent any accidental cleanup of source/CS tables when an execution is already in progress and an attempt to run the package by another user.

3.11. General Setup

- The input file format if any other than text files and Pipe delimited needs to be customized in Package configuration.
- The files in the output file path needs to be cleared after every run, otherwise files with same source name will be replaced.
- If control file is transmitted along with the source data files, procedure to consume the file and update SRC Audit file is available as part of UDA 's initial processing.

- The Control file layout needs to be discussed with client and SRC audit table design and SP needs to be changed according to client layout.

3.12. Error Reports

UDA captures the error records and error log in the below 2 process:

[Source Data upload to Source DB in UDA](#) – Any source records or files that have a layout different than the preset source layout in UDA will error out as part of Bulk Upload. There is a Reject count to capture the records in error during source data upload.

[Data type Validation in UDA](#) – Once the lookup and transformation is completed, records in CS_STG_* tables are validated for mandatory data and data type and length. Any records that fails the validation is errored out and logged in CS_ERR_ST_* tables.

Any record that fails the validation of mandatory fields for other applications like STARSERV,CLINICAL+,COHORT are captured as “Warning” and logged in CS_ERR_ST_* tables.

These error records are consolidated and produced as an Error Report at the end of current execution.

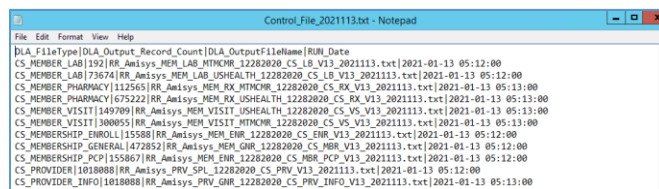
FILE_TYPE	FILE_NAME	ERROR_TYPE	ERROR_REASON	ERROR_COLUMN	ERROR_COUNT	RUN_DATE
CS_MEMBERSHIP_ENROLL	FLNEMR_ENROLLMENT_01.txt	Warning	Warning: Mandatory field for OTHER Applications missing.	Contract_ID	168276	1/13/2021 2:17
CS_MEMBERSHIP_ENROLL	FLNEMR_ENROLLMENT_02.txt	Warning	Warning: Mandatory field for OTHER Applications missing.	Contract_ID	168170	1/13/2021 2:17
CS_MEMBERSHIP_ENROLL	FLNEMR_ENROLLMENT_Test_01.txt	Warning	Warning: Mandatory field for OTHER Applications missing.	Contract_ID	173884	1/13/2021 2:17
CS_MEMBERSHIP_ENROLL	FLNEMR_ENROLLMENT_Test_02.txt	Warning	Warning: Mandatory field for OTHER Applications missing.	Contract_ID	168600	1/13/2021 2:17
CS_MEMBERSHIP_ENROLL	FLNEMR_ENROLLMENT_Test_03.txt	Warning	Warning: Mandatory field for OTHER Applications missing.	Contract_ID	164701	1/13/2021 2:17
CS_MEMBERSHIP_ENROLL	MEMR_ENROLLMENT_01.txt	Warning	Warning: Mandatory field for OTHER Applications missing.	Contract_ID	199932	1/13/2021 2:17
CS_MEMBERSHIP_ENROLL	MEMR_ENROLLMENT_02.txt	Warning	Warning: Mandatory field for OTHER Applications missing.	Contract_ID	199933	1/13/2021 2:17
CS_MEMBERSHIP_ENROLL	MEMR_ENROLLMENT_Test_01.txt	Warning	Warning: Mandatory field for OTHER Applications missing.	Contract_ID	200648	1/13/2021 2:17
CS_MEMBERSHIP_ENROLL	MEMR_ENROLLMENT_Test_02.txt	Warning	Warning: Mandatory field for OTHER Applications missing.	Contract_ID	200649	1/13/2021 2:17
CS_MEMBERSHIP_ENROLL	MEMR_ENROLLMENT_Test_03.txt	Warning	Warning: Mandatory field for OTHER Applications missing.	Contract_ID	400033	1/13/2021 2:17
CS_MEMBERSHIP_GENERAL	MEMR_GENERAL_20201204_MEMR_GNDRCD04_0101.txt	ERROR	ERROR: Mandatory field for HEDIS Application missing.	Gender	9	1/13/2021 2:17
CS_MEMBERSHIP_GENERAL	MEMR_GENERAL_20201204_MEMR_GNDRCD04_0102.txt	ERROR	ERROR: Mandatory field for HEDIS Application missing.	Gender	9	1/13/2021 2:17
CS_MEMBERSHIP_GENERAL	MEMR_GENERAL_20201204_MEMR_GNDRCD04_0103.txt	ERROR	ERROR: Mandatory field for HEDIS Application missing.	Gender	9	1/13/2021 2:17
CS_MEMBERSHIP_GENERAL	MEMR_GENERAL_20201204_MEMR_GNDRCD04_0104.txt	ERROR	ERROR: Mandatory field for HEDIS Application missing.	Gender	9	1/13/2021 2:17
CS_MEMBERSHIP_GENERAL	MEMR_GENERAL_20201204_MEMR_GNDRCD04_0105.txt	ERROR	ERROR: Mandatory field for HEDIS Application missing.	Gender	9	1/13/2021 2:17
CS_MEMBERSHIP_GENERAL	MEMR_GENERAL_20201204_MEMR_GNDRCD04_0106.txt	ERROR	ERROR: Mandatory field for HEDIS Application missing.	Rate_DataSource	4969	1/13/2021 2:17
CS_MEMBERSHIP_GENERAL	MEMR_GENERAL_20201204_MEMR_GNDRCD04_0106.txt	ERROR	ERROR: Mandatory field for HEDIS Application missing.	Rate_DataSource	4966	1/13/2021 2:17

The Error report can be customized to be transmitted to Client as per Client requirements.

Error Log – Any error that occurs during the execution of SSIS package is captured in Error Log and logged in CS_ERROR_LOG table for technical research and re-run.

3.13. Control File

UDA creates a control file after all the output files are generated. Control file contains the record count of each CS Output file and file name.



```

[DLA_FileType][DLA_Output_Record_Count][DLA_OutputFileName][RUN_Date]
CS_MEMBER_LAB|192|RR_AmIsys_MEM_LAB_MTHCHR_12282020_CS_LB_V13_20211113.txt|2021-01-13 05:12:00
CS_MEMBER_LAB|73674|RR_AmIsys_MEM_LAB_USHEALTH_12282020_CS_LB_V13_20211113.txt|2021-01-13 05:12:00
CS_MEMBER_PHARMACY|112565|RR_AmIsys_MEM_RX_MTHCHR_12282020_CS_RX_V13_20211113.txt|2021-01-13 05:13:00
CS_MEMBER_PHARMACY|675222|RR_AmIsys_MEM_RX_USHEALTH_12282020_CS_RX_V13_20211113.txt|2021-01-13 05:13:00
CS_MEMBER_VISIT|149789|RR_AmIsys_MEM_VISIT_USHEALTH_12282020_CS_VS_V13_20211113.txt|2021-01-13 05:13:00
CS_MEMBER_VISIT|198055|RR_AmIsys_MEM_VISIT_MTHCHR_12282020_CS_VS_V13_20211113.txt|2021-01-13 05:13:00
CS_MEMBERSHIP_ENROLL|15580|RR_AmIsys_MEM_EMR_12282020_CS_EMR_V13_20211113.txt|2021-01-13 05:12:00
CS_MEMBERSHIP_GENERAL|472852|RR_AmIsys_MEM_GHR_12282020_CS_MBR_V13_20211113.txt|2021-01-13 05:12:00
CS_MEMBERSHIP_PCP|155867|RR_AmIsys_MEM_PCP_12282020_CS_MBR_PCP_V13_20211113.txt|2021-01-13 05:12:00
CS_PROVIDER|1018808|RR_AmIsys_PRIV_SPL_12282020_CS_PRIV_V13_20211113.txt|2021-01-13 05:12:00
CS_PROVIDER_INFO|1018808|RR_AmIsys_PRIV_GHR_12282020_CS_PRIV_INFO_V13_20211113.txt|2021-01-13 05:13:00
  
```

Some of the clients require Load Order and some do not. The Control file can be customized as per Client requirements.

CS_CONTROL_FILE table contains the details of current run only for technical research and re-run. The control file details are archived before every run for tracking purpose.

3.14. Source Audit Report

UDA creates a Source Audit file after all the output files are generated. Source Audit file contains the audit status of all the input client files or tables and the processed/insert count and error count during the Source file/table upload.

SOURCE_FILETYPE	SOURCE_FILENAME	SOURCE_INPUT_RECORD_COUNT	SOURCE_REJECT_RECORD_COUNT	RUN_DATE
ENROLLMENT	*** Total Number of Records ***	167053	0	1/13/2021 5:11
LAB	*** Total Number of Records ***	476562	0	1/13/2021 5:11
MEM GENERAL	*** Total Number of Records ***	473701	0	1/13/2021 5:11
PHARMACY	*** Total Number of Records ***	739019	0	1/13/2021 5:11
PROVIDER	*** Total Number of Records ***	3054264	0	1/13/2021 5:11
VISIT	*** Total Number of Records ***	451038	0	1/13/2021 5:11
*** FILE LEVEL DETAILS AS FOLLOWS ***	*****	0	0	1/1/1900 0:00
ENROLLMENT	RR_Amisys_MEM_ENR_12282020.txt	167053	0	1/13/2021 5:11
LAB	RR_Amisys_MEM_LAB_MTMCMR_12282020.txt	1239	0	1/13/2021 5:11
LAB	RR_Amisys_MEM_LAB_USHEALTH_12282020.txt	475323	0	1/13/2021 5:11
MEM GENERAL	RR_Amisys_MEM_GNR_12282020.txt	473701	0	1/13/2021 5:11
PHARMACY	RR_Amisys_MEM_RX_MTMCMR_12282020.txt	105596	0	1/13/2021 5:11
PHARMACY	RR_Amisys_MEM_RX_USHEALTH_12282020.txt	634423	0	1/13/2021 5:11
PROVIDER	RR_Amisys_PRV_FACL_12282020.txt	1018088	0	1/13/2021 5:11
PROVIDER	RR_Amisys_PRV_GNR_12282020.txt	1018088	0	1/13/2021 5:11
PROVIDER	RR_Amisys_PRV_SPL_12282020.txt	1018088	0	1/13/2021 5:11
VISIT	RR_Amisys_MEM_VISIT_MTMCMR_12282020.txt	300001	0	1/13/2021 5:11
VISIT	RR_Amisys_MEM_VISIT_USHEALTH_12282020.txt	151037	0	1/13/2021 5:11

Source Audit report can be customized according to the client; transmission and format in the SSIS package to reflect the Audit and Balancing details the clients require.

SRC_AUDIT_FILE table contains the details of current run only for technical research and re-run. The control file details are archived before every run for tracking purpose.

3.15. Claims Count Report

UDA creates a Claims Count Report after all the output files are generated. Claims count report contains the distinct count of claims and membership depending on the Reporting Year configuration setup in ETL_Config table.

- 1) Medical Claims processed in the current run for the current measurement year/previous year
- 2) Pharmacy Claims processed in the current run for the current measurement year/previous year
- 3) Lab Claims processed in the current run for the current measurement year/previous year
- 4) Membership count or active lives processed in the current run for the current measurement year/previous year
- 5) Provider count processed in the current run for the current measurement year/previous year

ClaimType	FileName	ClaimsCount
All Claim Type	*****Total Claims Count*****	917120
LAB Summary	*****Total LAB Claims Count*****	42321
PHARMACY Summary	*****Total PHARMACY Claims Count*****	645471
VISIT Summary	*****Total VISIT Claims Count*****	229328
LAB	RR_Amisys_MEM_LAB_MTMCMR_12282020.txt	110
LAB	RR_Amisys_MEM_LAB_USHEALTH_12282020.txt	42211
PHARMACY	RR_Amisys_MEM_RX_MTMCMR_12282020.txt	12665
PHARMACY	RR_Amisys_MEM_RX_USHEALTH_12282020.txt	632806
VISIT	RR_Amisys_MEM_VISIT_MTMCMR_12282020.txt	79619
VISIT	RR_Amisys_MEM_VISIT_USHEALTH_12282020.txt	149709
ACTIVE MEMBER COUNT	RR_Amisys_MEM_ENR_12282020.txt	15526
MEMBER COUNT	RR_Amisys_MEM_GNR_12282020.txt	472852
PROVIDER COUNT	RR_Amisys_PRV_SPL_12282020.txt	1018088

Claims Count Report can be customized according to the client; transmission and format in the SSIS package to reflect the Audit and Balancing details the clients require.

4. System Requirements

Environments must meet the following requirements to install the application:

- Windows Enterprise Server 2012 and above
- Microsoft .NET Framework 4.5.1
- Required administrator access or equivalent permissions is mandatory
- Require a service account along with complete admin privileges

All other system requirements for ClaimSphere tool as mentioned in ClaimSphere Product Implementation Guide.

5. UDA Application Setup

Use this below procedures to setup the UDA application as an individual job, run through SQL Agent in SSMS.

5.1. Prerequisites

To install the UDA application:

- 1) Download the application kit from the file share site.
- 2) Extract the contents into the desired server folder.
- 3) Setup the connection with Client Side Database if source mode is Table
- 4) Setup the CS Destination tables
- 5) Fill out the Mapping configuration details from the Client
- 6) Setup the Configuration tables
- 7) Review if any custom procedures are required. If so, setup Custom Procedures.
- 8) Set up / Leverage existing File Transfer between Client and ClaimSphere environment.

5.2. Create Environment Folder

- 1) Unzip the DLA_Application_Deployer.zip
- 2) Copy DLA environment folder from
(*DLA_Application_Deployer\SQLObjects\Base\DLA_Environment*) paste into the
(*\\<<ServerName>>\R\$\ DLA_Environment*)
- 3) Rename the DLA_Environment folder as DB Name (*Eg., DLA_Environment -> DLA_PROS21*)
- 4) Change the server name and DB Name, etc in the dtsConfig Files *Under <<DB
Name>>\Configuration*

5.2.1. Configuring UDA _DLA_Master.dtsConfig, DLA_Slave.dtsConfig

Variables	Description	Example
\$(ServerName)	Server Name	10.28.161.200 or B55TESTSQL0002
\$(DB)	Database Name	DLA_PROS21
\$(DTSApplicationFilePath)	Binn Folder path	\\10.28.161.200\c\$\Program Files\Microsoft SQL Server\130\DTS\Binn
\$(EnvPath)	DLA_Environment path	\\10.28.161.200\r\$\

5.2.2. Configuring UDA _DLA_Slave_TableLoad.dtsConfig:

Variables	Description	Example
\$(DLA_ServerName)	DLA DB Server Name	10.28.161.200 or B55TESTSQL0002
\$(DLA_ServerName)	Source Tables Server Name	10.28.161.205 or B55TESTSQL0009
\$(DLA_DBName)	DLA Database Name	DLA_PROS21
\$(SRC_DBName)	Source Tables Database Name	UHG_NV_PROS20

5.3. Create Database

- 1) Connect the SQL Server Management Studio (SSMS)
- 2) Go to -- > Query --> SQLCMD Mode --> Execute
- 3) Deploy create database sql script available
[DLA_Application_Deployer\SQLObjects\Base\dladatabase.sql](#) (Update all required values in :setVar)

Variables	Description	Example
DB	Server Name	<i>:setvar DB DLA_PROS20</i>
ServerName	Database Name	<i>:setvar ServerName B55TESTSQL0002</i>
EnvPath	DLA_Environment path	<i>:setvar EnvPath "\\B55TESTSQL0002\r\$\\"</i>
UserId	User ID	<i>:setvar UserId "B55\cspsqltst.service"</i>
PkgPath	DTSX Path	<i>:setvar PkgPath "\\10.28.162.2\r\DLA_Application_Deployer\SQLObjects\Packages\"</i>
proxyname	Server Proxy name	<i>:setvar proxyname "UHG_SQL_Proxy"</i>

- 4) Deploy create msdb folder sql scripts available in [DLA_Application_Deployer\SQLObjects\Base\msdbfolder.sql](#) (Update all required values in *:setVar*)
- 5) Deploy create sql agent job creation scripts available in [DLA_Application_Deployer\SQLObjects\Job](#)

5.4. SQL Objects Deployment

- 1) Change the server name and DB Name in the dtsConfig Files under [DLA_Application_Deployer\Configuration\](#)

Variables	Description	Example
\$(ServerName)	Server Name	10.28.161.200
\$(DB)	Database Name	DLA_PROS21
\$(FilePath)	SQL Objects path	\\10.28.162.2\r\DLA_Application_Deployer\SQLObjects\

- 2) Run the bat file available in [DLA_Application_Deployer\BatchScript\DLA_Deployer.bat](#)
- 3) If any Issues reported in bat file ref. the table [\[dbo\].\[DeployerErrorLog\]](#)
(To verify deployed objects list Ref the tables [\[dbo\].\[ModuleHistory\]](#), [\[dbo\].\[ObjectsHistory\]](#))
- 4) If any issue in objects try with manual deployment of particular objects

5.5. SSIS Packages Deployment

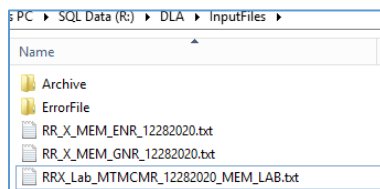
- 1) Deploy the Import ssis packages sql scripts available in [DLA_Application_Deployer\SQLObjects\Packages\dlaPackagesImport.sql](#) (Update all required values in *:setVar*)-> Ref Step 2

- 2) Deploy SSIS Packages table metadata available in
DLA_Application_Deployer\SQLObjects\Packages\dla_m_packagelist.sql (Update all required values in :setVar) ->Ref Step 2
- 3) Go to -- > Query --> SQLCMD Mode --> Execute

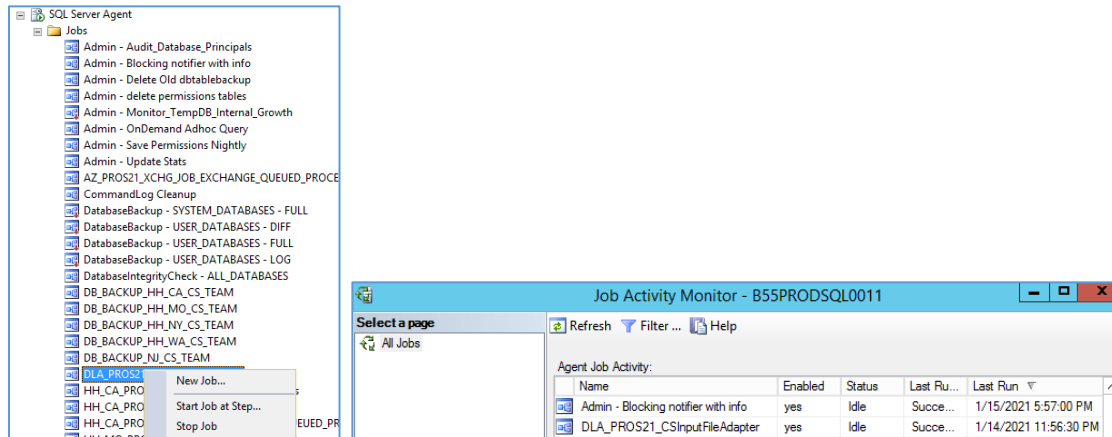
5.6. Execution - UDA Application

Use the below procedure to execute the application.

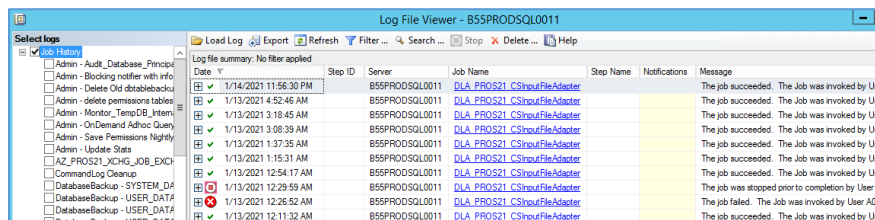
Place the Client provided Control files (if any) and Input data files in the Sourcefilepath folder as per the configuration setup.



By double clicking the Run Job in the Integration Services, execution is started.



The job history can be viewed through :



5.7. Job Progress

The current run status of the job can be viewed in SSMS query window by viewing Process status DLA.JOBRUN_MASTER table for the current run.

RunId	StartDate	EndDate	Process_Status	UserId
1	2021-01-12 05:07:05	2021-01-12 05:09:31	Failed	b55\cspsqlprod.service
2	2021-01-12 05:21:40	2021-01-12 05:21:58	Completed	b55\cspsqlprod.service
3	2021-01-12 05:23:44	2021-01-12 05:25:58	Completed	b55\cspsqlprod.service
4	2021-01-13 05:14:58	2021-01-13 10:41:52	Completed	b55\cspsqlprod.service
5	2021-01-14 23:52:20	2021-01-15 02:18:13	Completed	b55\cspsqlprod.service

The status of individual packages, can also be viewed in SSMS query window by viewing Processstatus column in DLA .DLA_PACKAGELIST table. Individual packages go through the status:



Any failures/logging can be viewed in REPORT_LOG, ERROR_LOG, PACKAGEERRORLOG tables for debugging.

Output files can be viewed in Output files folder. It contains the data intake files along with Control file, Error file, Claims Count and Source Audit Report for the current run.

The Output files will have to be transferred to the server where the ClaimSphere app is setup to receive files. This can be customized depending on the customer and CS server.

6. Annexure



HEDIS_Input_File_Specifications_V12.2_



Client - CS file

mapping_V1.1.xlsm

7. Glossary

S.No	Term	Explanation
1	Data Intake Files	Set of data files originating from Client/Health plan containing information related to health plan members, providers and claims to compute HEDIS and State Measure rate.
2	Client	Health Plan / Health Insurance companies / Payer who are new customers, using ClaimSphere HEDIS tool

3	UDA	Universal Data Adaptor – Application developed to convert source data files to ClaimSphere specific data intake files.
4	Source data files	Data files originating from Client side that do not necessarily follow ClaimSphere intake file layouts.
5	SSIS	SQL Server Integration Services – ETL tool
6	Mapping	Mapping, in this document is used to denote the DB setup of source side field equivalent to field in ClaimSphere formatted files.
7	CS DB	Set of tables in UDA application that has exact layout of Staging tables present in HEDIS ClaimSphere (HDS) Tool.
8	Lookup	Some source data would require conversion to ClaimSphere intake format. Example – Claim Status could have been captured as P (Paid), D (Denied) at source end, but ClaimSphere accepts only 1 & 2. A lookup directory is required and values are translated as per the documented values.
9	Transformation	Some source details have values concatenated together, some are stored as separated values. Some source values may have default date/ open ended term date stored as 2199-12-31. ClaimSphere requires all these values converted to the format that is specified in Input file specifications.
10	Cleansing	Set of Cleansing rules setup in ClaimSphere application that will eliminate any records that do not have expected CS format, absence of mandatory fields, value, invalid date or numeric formats.

© Copyright 2020, Cognizant

All rights reserved. No part of this document may be reproduced, stored in a retrieval system, transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the express written permission from Cognizant. The information contained herein is subject to change without notice. All other trademarks mentioned herein are the property of their respective owners.

Cognizant (NASDAQ-100: CTSI) is one of the world's leading professional services companies, transforming clients' business, operating and technology models for the digital era. Our unique industry-based, consultative approach helps clients envision, build and run more innovative and efficient businesses. Headquartered in the U.S., Cognizant is ranked 195 on the Fortune 500 and is consistently listed among the most admired companies in the world. Learn how Cognizant helps clients lead with digital at www.cognizant.com or follow us @Cognizant.



**World
Headquarters**

500 Frank W.
Burr Blvd.
Teaneck, NJ
07666 USA
Phone: +1 201
801 0233
Fax: +1 201
801 0243
Toll Free: +1
888 937 3277
Email:
inquiry@cogni
zant.com

**European
Headquarters**

1 Kingdom
Street
Paddington
Central
London W2
6BD
Phone: +44 (0)
207 297 7600
Fax: +44 (0)
207 121 0102
Email:
infouk@cogniz
ant.com

India

**Operations
Headquarters**

#5/535, Old
Mahabalipuram
Road
Okkiyam Pettai,
Thoraipakkam
Chennai, 600
096 India
Phone: +91 (0)
44 4209 6000
Fax: +91 (0) 44
4209 6060
Email: inquiryindi
a@cognizant.co
m

Visit us online at www.cognizant.com or follow us on Twitter: Cognizant.

